

Workshop Registration Service

Notes on how I built it

Stack

I used React with Vite and axios on the frontend, and Node, Express, Prisma and MySQL on the backend. I added Zod to validate every request, and Tailwind so I could get a clean, simple screen quickly. I kept the dependencies few on purpose, so the code stays easy for the next person to pick up.

Stopping overbooking

This was the part I cared about most. Every change to seats (registering, cancelling, moving someone off the waitlist, changing capacity) happens inside one database transaction that first locks that workshop's row. If two people try to take the last seat at the same moment, MySQL makes one wait, and the second finds the workshop full and is told so. Different workshops don't block each other.

As a safety net, the database itself has a rule that seats taken can never go above capacity, so even a mistake in my code couldn't overbook. I also wrote a script that fires lots of requests at once, and checked that exactly the right number get a seat.

Main decisions

- Permissions are checked in the backend, in one place. The user is looked up on every request, so removing someone's access takes effect straight away. The frontend only hides buttons.
- Nothing is ever deleted. A cancelled registration stays, with who cancelled it, when and why. Staff accounts are deactivated, not removed.
- "Full" isn't stored as a status. It is worked out from capacity and seats taken, so it can't go out of sync.
- Extra things I track: location, end time, description, who created or edited each workshop, and a cancel reason.
- I did both bonus items: an audit trail of changes, and a waitlist that gives a freed seat to whoever has waited longest.

Assumptions

- An attendee is just a name and email, and the same email can't hold two bookings in one workshop.
- Bookings are only allowed while a workshop is open and hasn't started.
- Staff will contact waitlisted attendees themselves when a seat opens, since attendees have no accounts.

Trade-offs

- Keeping a seat counter on each workshop means one more number to keep correct, but it makes the capacity check and the "has seats" filter fast. The lock and the database rule protect it.
- Locking per workshop favours being correct over being fast. For about 15 staff, that is the right choice.
- The login token is stored in the browser for simplicity. A secure cookie would be better for production.

What I skipped

- Emails or texts to attendees.
- A screen to manage locations (they are set up in the seed data).
- Password reset by email. Admins set passwords instead.
- Per-location timezones. Times are shown in the browser's timezone.
- A full automated test suite. I only tested the concurrency behaviour, as it is the riskiest part.